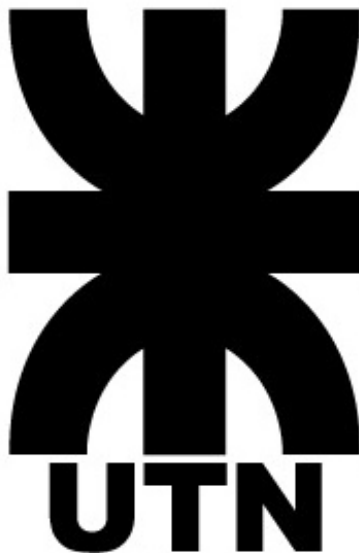




Universidad Tecnológica Nacional

Facultad Regional Tucumán

Trabajo Final Integrador

Blog Personal sobre infraestructura Proxmox

Materia: Virtualización: Consolidación de Servidores

Comisión: 5K3

Profesor: Ing. Carriles, Luis María

Autor

Nombre: Armas, Mauro Nahuel

DNI: 44.581.626

Legajo: 53812

Índice

1. Introducción	2
1.1. Resumen ejecutivo de la solución	2
1.2. Sobre el tono de este informe	2
2. Marco conceptual: Proxmox VE como hipervisor	3
2.1. Qué es Proxmox y por qué se eligió	3
2.2. El motor de cómputo dual: KVM vs LXC	3
2.3. Capacidades core relevantes para el trabajo	3
3. Diseño e implementación de la infraestructura	4
3.1. Especificaciones técnicas adoptadas	4
3.2. Procedimiento de creación de contenedores	4
4. Desarrollo de la aplicación Web	5
4.1. Elección del stack tecnológico	5
4.2. Estructura del proyecto	5
4.3. Configuración crítica: modo standalone	5
4.4. Esquema de base de datos	6
5. Despliegue paso a paso	7
5.1. Fase 1: PostgreSQL en el contenedor de base de datos	7
5.2. Fase 2: Preparación del build en entorno local	7
5.3. Fase 3: Transferencia del código	8
5.4. Fase 4: Servicio systemd persistente	8
5.5. Fase 5: Resolución del error de permisos PostgreSQL	9
6. Exposición externa: Cloudflare Tunnel	10
6.1. El problema	10
6.2. Mecanismo de funcionamiento	10
6.3. Implementación	10
7. Verificación funcional y consumo de recursos	11
7.1. Pruebas funcionales realizadas	11
7.2. Consumo de recursos en producción	11
8. Conclusión personal	11
9. Glosario de términos clave	12

1. Introducción

El presente informe documenta el proceso de desarrollo, implementación y despliegue de un **Blog Personal** sobre la infraestructura de virtualización de la NAP (Nube Académica Personal) de la UTN-FRT, accesible mediante nap.frt.utn.edu.ar.

El trabajo no se limitó a aplicar una receta: cada decisión técnica fue producto de un análisis previo, en algunos casos descartando alternativas más cómodas pero menos didácticas. El objetivo personal fue tratar el TPF como un ejercicio real de administración de sistemas en entornos restringidos, donde el desafío principal no es el código sino la disciplina para operar dentro de límites estrictos de recursos.

1.1 Resumen ejecutivo de la solución

La arquitectura desplegada consiste en dos contenedores LXC independientes sobre Proxmox VE, comunicados mediante red interna en el segmento privado 172.16.90.0/24:

- **Contenedor 159 (44581626A):** aplicación web desarrollada en **Next.js 16** (React + Node.js) en modo *standalone* para minimizar consumo.
- **Contenedor 162 (44581626DB):** motor de base de datos **PostgreSQL 15** optimizado para 128 MB de RAM.

Ambos contenedores operan dentro del estricto límite de 128 MB de RAM y 8 GB de disco impuesto por la cátedra. Para la exposición externa se utilizó **Cloudflare Tunnel**, un mecanismo de tunelización reversa que evita los problemas perimetrales de la red universitaria.

1.2 Sobre el tono de este informe

A diferencia de un manual de procedimientos clásico, este documento intercala explicaciones de los porqués detrás de cada decisión, los errores encontrados durante el proceso y cómo se resolvieron.

2. Marco conceptual: Proxmox VE como hipervisor

Antes de entrar en el despliegue propiamente dicho, conviene fijar los conceptos teóricos que sustentan la arquitectura. Esta sección se apoya en el material visto en clase y en bibliografía complementaria sobre el ecosistema Proxmox.

2.1 Qué es Proxmox y por qué se eligió

Proxmox Virtual Environment (PVE) es una plataforma de virtualización de código abierto que combina dos motores de cómputo bajo una misma interfaz: **KVM** para máquinas virtuales completas y **LXC** para contenedores Linux ligeros. Un punto fundamental que vale la pena subrayar: Proxmox **no se instala como una aplicación** sobre un sistema operativo existente. Proxmox es, en sí mismo, el sistema operativo (basado en Debian Linux). Al instalarse, toma control total del hardware, lo que le permite acceso directo al kernel y a los recursos físicos.

Las tecnologías que respaldan a Proxmox son estándares maduros de la industria: KVM, LXC, ZFS, Ceph, Linux estándar. Esto significa que no existe *vendor lock-in* y que los conocimientos adquiridos son transferibles a cualquier entorno Linux profesional.

2.2 El motor de cómputo dual: KVM vs LXC

La elección entre VM y contenedor no es trivial y determina recursos, rendimiento y casos de uso. La cátedra plantea esta dualidad como punto central de la materia.

Aspecto	KVM (VM)	LXC (Contenedor)
Aislamiento	Hardware virtualizado completo	Nivel SO (comparte kernel)
Compatibilidad	Cualquier SO (Windows, BSD, Linux)	Solo distribuciones Linux
Consumo de RAM	Alto (cada VM emula CPU y RAM)	Mínimo (apenas mayor al proceso)
Arranque	Decenas de segundos	Uno o dos segundos
Caso de uso ideal	Sistemas heredados, aislamiento estricto	Microservicios, alta densidad

Para este TPF se eligió LXC. La consigna imponía un límite de 128 MB de RAM por contenedor, lo cual hace inviable una VM completa: solo el arranque de un sistema operativo invitado consumiría más de ese límite. LXC, en cambio, comparte el kernel del host Proxmox y permite que prácticamente toda la RAM disponible sea utilizada por la aplicación efectiva.

2.3 Capacidades core relevantes para el trabajo

De las capacidades que ofrece Proxmox, las que tuvieron impacto directo en el TPF fueron:

- **Almacenamiento avanzado:** aunque no se manipuló directamente, los contenedores residen en almacenamiento LVM-Thin del clúster universitario, lo que permite *thin provisioning* (el espacio se asigna conforme se usa).
- **Redes Definidas por Software (SDN):** la interfaz `eth0` de cada contenedor se conectó al bridge virtual `vbr0` de Proxmox, que actúa como switch interno. Sobre este bridge opera la red privada `172.16.90.0/24`.
- **Diseño multi-master:** cualquier nodo del clúster puede administrarse desde la interfaz web de cualquier otro. Esto se vuelve relevante a la hora de gestionar varios contenedores desde un solo punto.

3. Diseño e implementación de la infraestructura

3.1 Especificaciones técnicas adoptadas

Siguiendo las pautas de la cátedra (uso del DNI como base para el hostname, ID del contenedor reflejado en el último octeto de la IP), se aprovisionaron dos contenedores con los siguientes parámetros:

Componente	Servidor App (LXC 159)	Servidor BD (LXC 162)
Hostname	44581626A	44581626DB
Plantilla OS	Debian 12 standard	Debian 12 standard
vCPU	1 core	1 core
Memoria RAM	128 MB	128 MB
Swap	128 MB	128 MB
Disco	8 GB (ext4)	8 GB (ext4)
Bridge de red	vmbr0	vmbr0
Dirección IP	172.16.90.159/24	172.16.90.162/24
Gateway / DNS	172.16.90.1	172.16.90.1

3.2 Procedimiento de creación de contenedores

Los contenedores se crearon desde la interfaz web de Proxmox (<https://nap.frt.utn.edu.ar/v1:0:18:4:>) con autenticación 2FA TOTP previamente configurada para el usuario. El flujo seguido fue:

1. Autenticación en el portal y verificación TOTP con app Google Authenticator.
2. Botón **Create CT** en la esquina superior derecha del panel.
3. Pestaña *General*: asignación de CT ID, hostname y contraseña del usuario 44581626.
4. Pestaña *Template*: selección de `debian-12-standard_12.7-1_amd64.tar.zst`.
5. Pestaña *Disks*: 8 GB en almacenamiento `local-lvm`.
6. Pestaña *CPU y Memory*: 1 core, 128 MB de RAM, 128 MB de swap.
7. Pestaña *Network*: bridge `vmbr0`, IPv4 static con CIDR `172.16.90.159/24` y gateway `172.16.90.1`.
8. Pestaña *DNS*: DNS servers `172.16.90.1` y DNS domain vacío.

Decisión sobre tipo de IP

Aunque era posible utilizar DHCP, opté por IP estática en ambos contenedores. La razón es práctica: la cadena de conexión de la aplicación al motor de base de datos exige una dirección estable. Una IP que cambie por renovación DHCP rompería el servicio en silencio, sin avisos. Sacrificar dinamismo a cambio de previsibilidad fue una decisión consciente.

4. Desarrollo de la aplicación Web

4.1 Elección del stack tecnológico

Para la capa de aplicación se optó por **Next.js 16** sobre Node.js 20. La decisión se justifica por varios motivos:

- **Modo *standalone*:** Next.js permite generar un build autocontenido que incluye solo las dependencias estrictamente necesarias para producción. Esto reduce drásticamente el footprint, ideal para los 128 MB de RAM disponibles.
- **Renderizado en servidor (SSR):** las páginas se generan en el servidor antes de enviarse al cliente, ofreciendo mejor rendimiento percibido y SEO sin requerir JavaScript adicional en el navegador.
- **Estructura clara:** el framework impone convenciones de rutas y separación de componentes que facilitan el mantenimiento.
- **Familiaridad personal:** es una tecnología que vengo utilizando en otros proyectos personales, lo que me permitió enfocarme en los aspectos de infraestructura del TPF en lugar de pelearme con un framework nuevo.

4.2 Estructura del proyecto

El blog está organizado en una estructura típica del App Router de Next.js:

```
PORTAFOLIO/
|-- .next/                # build de produccion
|-- public/
|   |-- mauro.jpeg        # foto personal
|   |-- informe-tpf.pdf   # este documento
|-- src/
|   |-- app/
|       |-- admin/        # CRUD de posts (autenticado)
|       |-- api/          # endpoints de Next.js
|       |-- posts/[slug]/ # detalle dinamico de cada post
|       |-- page.js       # home del blog
|       |-- layout.js     # layout raiz
|   |-- components/      # componentes reutilizables
|   |-- lib/db.js         # cliente PostgreSQL
|-- init.sql             # schema inicial de la DB
|-- next.config.mjs      # config con output: standalone
|-- package.json
```

4.3 Configuración crítica: modo standalone

El archivo `next.config.mjs` contiene la directiva clave que hace viable el despliegue en 128 MB:

```
const nextConfig = {
  output: 'standalone',
  compress: true,
  experimental: {
    serverMinification: true
  }
};
```

Esta configuración genera, al ejecutar `npm run build`, una carpeta `.next/standalone` que contiene el servidor Node.js mínimo necesario para servir la aplicación, sin `node_modules` de desarrollo.

4.4 Esquema de base de datos

La persistencia se modeló sobre una única tabla `posts` que captura tanto los metadatos como el contenido de cada artículo:

```
CREATE TABLE IF NOT EXISTS posts (  
  id          SERIAL PRIMARY KEY,  
  slug        VARCHAR(255) NOT NULL UNIQUE,  
  title       VARCHAR(255) NOT NULL,  
  excerpt     TEXT,  
  content     TEXT,  
  tag         VARCHAR(100),  
  status      VARCHAR(50) NOT NULL DEFAULT 'draft',  
  read_time   INTEGER,  
  published_at TIMESTAMP WITH TIME ZONE,  
  created_at  TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  updated_at  TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

El uso de `SERIAL PRIMARY KEY` y `TIMESTAMP WITH TIME ZONE` es propio de PostgreSQL, lo que descartó MySQL/MariaDB como alternativas.

5. Despliegue paso a paso

5.1 Fase 1: PostgreSQL en el contenedor de base de datos

Una vez ingresado al contenedor 162 (44581626DB) mediante la consola web de Proxmox, se ejecutó la instalación del motor de base de datos:

```
apt-get update
apt-get install -y postgresql postgresql-contrib
```

Optimización para entorno de bajos recursos. La configuración por defecto de PostgreSQL asume varios GB de RAM disponibles. Para los 128 MB del contenedor fue necesario reducir agresivamente los buffers internos:

```
sed -i "s/^#shared_buffers = 128MB/shared_buffers = 16MB/" \
/etc/postgresql/15/main/postgresql.conf
sed -i "s/^#work_mem = 4MB/work_mem = 2MB/" \
/etc/postgresql/15/main/postgresql.conf
```

Habilitación de conexiones remotas. Por defecto PostgreSQL solo acepta conexiones desde localhost. Como la aplicación corre en otro contenedor, fue necesario abrir la escucha en todas las interfaces:

```
sed -i "s/.*listen_addresses.*/listen_addresses = '*'/" \
/etc/postgresql/15/main/postgresql.conf
```

Reglas de autenticación host. Se restringió el acceso remoto exclusivamente a la IP del contenedor de aplicación, descartando autenticar a cualquier otro host de la red:

```
echo "host all all 172.16.90.159/32 scram-sha-256" \
>> /etc/postgresql/15/main/pg_hba.conf
systemctl restart postgresql
```

Creación del usuario y la base de datos:

```
su - postgres
psql -c "CREATE USER mauro WITH PASSWORD 'mauro';"
psql -c "CREATE DATABASE blogdb OWNER mauro;"
```

5.2 Fase 2: Preparación del build en entorno local

La compilación del proyecto Next.js se realizó en mi equipo personal (Linux, i3-8130U, 24 GB RAM) ya que requiere herramientas de desarrollo que no son viables dentro de los 128 MB del contenedor:

```
cd ~/Proyectos/Portafolio
npm run build
```

Esta operación generó la carpeta `.next/standalone` con el servidor autocontenido.

5.3 Fase 3: Transferencia del código

Obstáculo encontrado y solución

El intento inicial de transferir el código vía SCP desde mi equipo a la IP 172.16.90.159 falló con `Connection timed out`. La causa: la red 172.16.90.0/24 es **privada interna de la universidad**, no ruteable desde el exterior. Esto implica que ningún cliente SSH desde fuera de la red de la UTN puede conectarse directamente a los contenedores.

Solución adoptada: usar GitHub como repositorio intermedio. El contenedor sí tiene salida HTTPS hacia internet (puerto 443), por lo que puede clonar repositorios públicos. Este enfoque tiene una ventaja adicional sobre el SCP: deja un registro versionado del estado exacto del código desplegado, útil para auditar o revertir cambios.

El flujo definitivo de despliegue del código fue:

```
# En el equipo local (Zorin OS):
git add -f .next/standalone .next/static public
git commit -m "deploy: build produccion para contenedor UTN"
git push origin main
```

```
# En la consola del contenedor 159 (44581626A):
apt-get update && apt-get install -y curl git
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt-get install -y nodejs

mkdir -p /var/www/blog
git clone https://github.com/mauroarmas/blogMauro.git /tmp/repo
cp -r /tmp/repo/.next/standalone/. /var/www/blog/
cp -r /tmp/repo/public /var/www/blog/public
cp -r /tmp/repo/.next/static /var/www/blog/.next/static
```

5.4 Fase 4: Servicio systemd persistente

Ejecutar `node server.js` manualmente desde la consola funciona, pero el proceso muere al cerrar la sesión. Para garantizar que el blog permanezca activo y se reinicie automáticamente ante fallos, se creó un servicio systemd dedicado:

```
cat > /etc/systemd/system/blog.service << 'EOF'
[Unit]
Description=Blog Next.js - Mauro Armas
After=network.target

[Service]
Type=simple
WorkingDirectory=/var/www/blog
Environment=PORT=3000
Environment=NODE_ENV=production
Environment=DATABASE_URL=postgres://mauro:mauro@172.16.90.162:5432/blogdb
ExecStart=/usr/bin/node server.js
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF
```

```
systemctl daemon-reload
systemctl enable blog
systemctl start blog
```

La directiva `Restart=always` con `RestartSec=3` hace que, ante cualquier caída del proceso Node.js, systemd lo reinicie automáticamente en 3 segundos, esto es relevante pues ningún proceso queda atado a una terminal abierta.

5.5 Fase 5: Resolución del error de permisos PostgreSQL

Al levantar el servicio por primera vez, el blog cargaba la página de inicio pero no mostraba ningún post. Los logs revelaron el error real:

```
journalctl -u blog -n 30 --no-pager
# ...
# code: '42501',
# severity: 'ERROR',
# file: 'aclchk.c',
# routine: 'aclcheck_error'
```

El código PostgreSQL 42501 significa *permission denied*. El usuario **mauro** podía conectarse a la base **blogdb**, pero no tenía permisos sobre la tabla **posts**. ¿La causa? La tabla había sido creada por el usuario **postgres** (no por **mauro**) durante las pruebas iniciales, y aunque **mauro** era propietario de la base de datos, no heredaba automáticamente la propiedad de los objetos creados por otros usuarios dentro de ella.

Solución aplicada desde el contenedor de base de datos:

```
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO mauro;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO mauro;
ALTER TABLE posts OWNER TO mauro;
```

Este episodio fue formativo: refleja una sutileza del modelo de permisos de PostgreSQL que no es evidente en la documentación introductoria. Ser dueño de una base de datos no implica ser dueño de sus objetos internos.

6. Exposición externa: Cloudflare Tunnel

6.1 El problema

Con el blog corriendo en `http://172.16.90.159:3000` dentro de la red privada de la UTN, surgió el mismo desafío que enfrenta cualquier despliegue en entornos universitarios: **cómo permitir que cualquier usuario acceda al blog desde fuera de la universidad.**

Las opciones consideradas fueron:

1. **Port forwarding tradicional:** requiere intervención del área de redes de la UTN, fuera del alcance.
2. **VPN universitaria:** la institución no provee este servicio para usuarios estudiantes.
3. **Cloudflare Tunnel:** servicio gratuito de Cloudflare que establece un túnel saliente desde el contenedor hacia su red global, exponiendo el servicio sin abrir puertos entrantes.

Se optó por **Cloudflare Tunnel** por tres razones: es gratuito, no requiere cuenta para el modo *quick tunnel*, y opera sobre HTTPS (puerto 443), tráfico que los firewalls perimetrales suelen permitir sin restricciones.

6.2 Mecanismo de funcionamiento

Cloudflare Tunnel invierte el modelo de exposición tradicional. En lugar de que el cliente externo intente conectarse al servidor (lo cual exige abrir puertos), es el servidor el que inicia una conexión **saliente** hacia la red de Cloudflare. Esta conexión se mantiene abierta y actúa como túnel: las peticiones externas llegan a Cloudflare con una URL pública del dominio `trycloudflare.com`, y Cloudflare las reenvía por el túnel preestablecido hacia el blog.

Como el tráfico viaja sobre TLS por el puerto 443, los sistemas perimetrales lo interpretan como navegación HTTPS normal y no lo bloquean.

6.3 Implementación

```
curl -L https://github.com/cloudflare/cloudflared/releases/latest/\
download/cloudflared-linux-amd64 -o /usr/local/bin/cloudflared
chmod +x /usr/local/bin/cloudflared

cloudflared tunnel --url http://localhost:3000
```

Al ejecutar el comando, cloudflared establece la conexión hacia la red de Cloudflare y devuelve una URL pública del tipo `https://*.trycloudflare.com`. El blog queda accesible desde cualquier punto de internet, con certificado HTTPS válido proporcionado por Cloudflare.

Limitación de los quick tunnels

El modo *quick tunnel* (sin cuenta de Cloudflare) genera una URL aleatoria distinta en cada ejecución, válida solo mientras el proceso permanezca activo. Para la exposición se reinicia el túnel inmediatamente antes de la evaluación y se comparte la URL en ese momento. Para un entorno productivo real, se utilizaría *Cloudflare Tunnel* con cuenta autenticada y un dominio propio, lo que provee URL permanente y configuración persistente como servicio systemd.

7. Verificación funcional y consumo de recursos

7.1 Pruebas funcionales realizadas

- **Conexión de red entre contenedores:** verificada con `nc -zv 172.16.90.162 5432`, retornando `Connection succeeded!`.
- **Listado de posts:** la página principal del blog muestra correctamente la lista de artículos publicados desde la base de datos.
- **CRUD completo:** desde el panel `/admin` se probó crear, editar, marcar como borrador, publicar y eliminar posts. Todas las operaciones se reflejan en la base de datos.
- **Persistencia ante reinicios:** se reinició el contenedor 159 y el servicio `systemd` levantó el blog automáticamente sin intervención manual.
- **Acceso externo:** se probó la URL pública de Cloudflare desde una red domiciliaria distinta, confirmando accesibilidad fuera de la red UTN.

7.2 Consumo de recursos en producción

Una vez en operación estable, se midió el consumo real de los procesos principales:

Contenedor	Proceso principal	RAM utilizada
LXC 159 (44581626A)	Next.js (Node.js v20)	≈ 80 MB
LXC 162 (44581626DB)	PostgreSQL 15	≈ 45 MB

Ambos contenedores operan holgadamente por debajo del límite de 128 MB, dejando margen suficiente para picos de carga durante la defensa.

8. Conclusión personal

Este trabajo representó mucho más que desplegar una aplicación web sobre dos contenedores. Fue un ejercicio de administración de sistemas reales. La limitación de 128 MB de RAM, que inicialmente parecía un obstáculo arbitrario, terminó por definir la arquitectura completa: obligó a descartar KVM en favor de LXC (porque una máquina virtual completa consumiría más que ese límite solo para arrancar), a compilar Next.js en modo *standalone* (eliminando cientos de megabytes de dependencias de desarrollo), y a reconfigurar PostgreSQL reduciendo sus buffers internos a valores que en un entorno productivo real serían insuficientes pero que aquí resultaron perfectamente funcionales. Cada decisión técnica tuvo un porqué derivado directamente de esa restricción.

El bloqueo perimetral de la red universitaria introdujo un problema que no aparece en ningún tutorial: ¿cómo se accede a un servicio que corre en una red privada inaccesible desde el exterior? La respuesta, un túnel reverso mediante Cloudflare Tunnel, me llevó a comprender un concepto fundamental de redes que la teoría presenta como abstracto pero que aquí fue tangible: la diferencia entre conexiones entrantes (bloqueadas por el firewall institucional) y conexiones salientes (permitidas por el puerto 443), y cómo invertir el modelo de exposición para que sea el servidor quien inicie la comunicación hacia afuera.

Trabajar con recursos limitados es, una experiencia más enriquecedora que trabajar con recursos ilimitados. Cuando todo abunda, las decisiones técnicas se toman por conveniencia. Cuando cada megabyte cuenta, se toman por necesidad, y esas son las que realmente se internalizan.

9. Glosario de términos clave

Bridge virtual

Switch de red implementado en software dentro del hipervisor. En Proxmox, `vbr0` es el bridge por defecto que conecta los contenedores entre sí y con la red externa.

Build standalone (Next.js)

Modo de compilación que genera un servidor Node.js autocontenido, incluyendo solo las dependencias estrictamente necesarias para producción. Reduce el tamaño del despliegue.

Cloudflared

Cliente oficial de Cloudflare que establece túneles salientes hacia la red de Cloudflare, permitiendo exponer servicios sin abrir puertos entrantes.

Contenedor LXC

Linux Container. Forma de virtualización a nivel de sistema operativo que comparte el kernel con el host y aísla procesos, archivos y red mediante *namespaces* y *cgroups*.

Hipervisor de tipo 1

Software de virtualización que se instala directamente sobre el hardware (bare-metal), sin un sistema operativo intermedio. Proxmox VE y VMware ESXi son ejemplos.

KVM

Kernel-based Virtual Machine. Módulo del kernel Linux que convierte al sistema operativo en hipervisor tipo 1 y permite ejecutar VMs con sistemas operativos completos.

Pmxdfs

Proxmox Cluster File System. Sistema de archivos replicado en RAM que mantiene sincronizada la configuración entre todos los nodos de un clúster Proxmox.

Quick tunnel

Modo de Cloudflare Tunnel que crea un túnel temporal sin requerir cuenta de Cloudflare. Útil para demostraciones, no para producción.

Scram-sha-256

Método de autenticación de PostgreSQL basado en challenge-response. Más seguro que MD5 y que el método *peer* usado por defecto en conexiones locales.

SDN

Software-Defined Networking. Redes gestionadas por software, donde topología, VLANs y reglas se configuran desde una interfaz centralizada en lugar de en hardware dedicado.

Systemd

Sistema de inicio y gestor de servicios usado por Debian (y por la mayoría de distribuciones modernas). Mantiene procesos vivos, gestiona dependencias entre servicios y reinicia automáticamente ante fallos.

TOTP

Time-based One-Time Password. Algoritmo que genera códigos de 6 dígitos que cambian cada 30 segundos. Usado como segundo factor de autenticación en la plataforma Proxmox de la UTN.